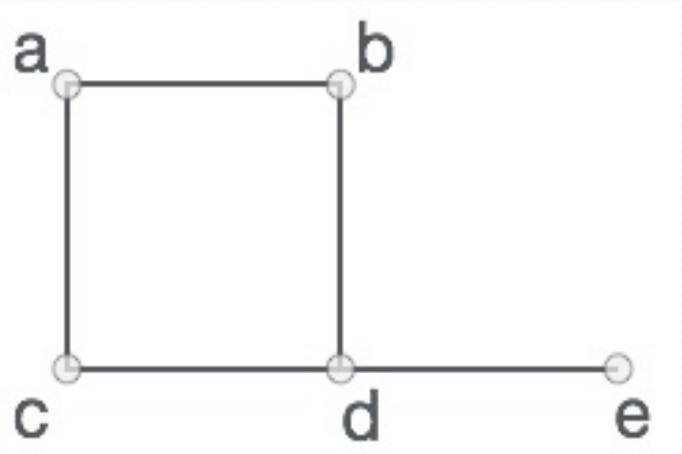
Advanced Data Structure

Graph Data Structure

Graph

- A graph is a pictorial representation of a set of objects where some pairs of objects are connected by links.
- The interconnected objects are represented by points termed as **vertices**, and the links that connect the vertices are called **edges**.
- Formally, a graph is a pair of sets **(V, E)**, where **V** is the set of vertices and **E** is the set of edges, connecting the pairs of vertices. Take a look at the following graph –



In this graph,

V = {a, b, c, d, e}

E = {ab, ac, bd, cd, de}

Types of Graphs:

- **Undirected Graphs:**

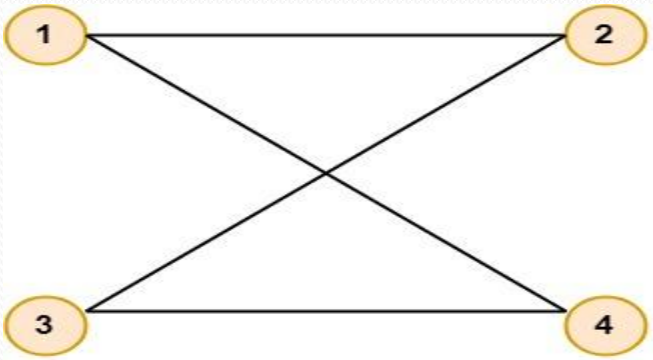


Fig:1

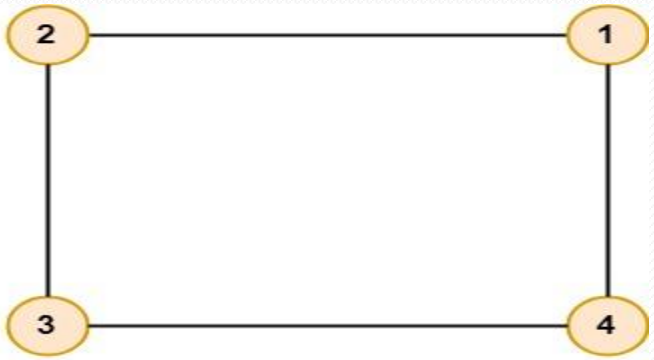
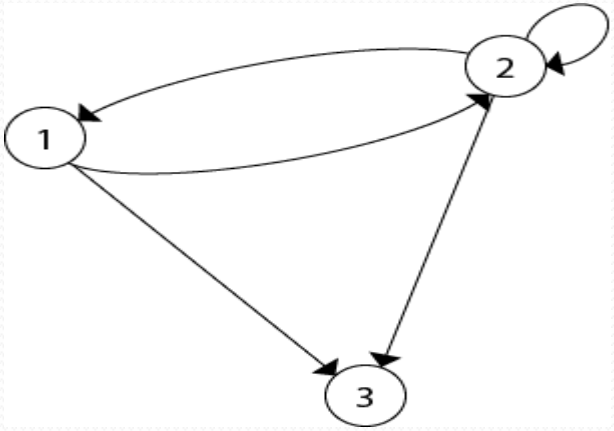


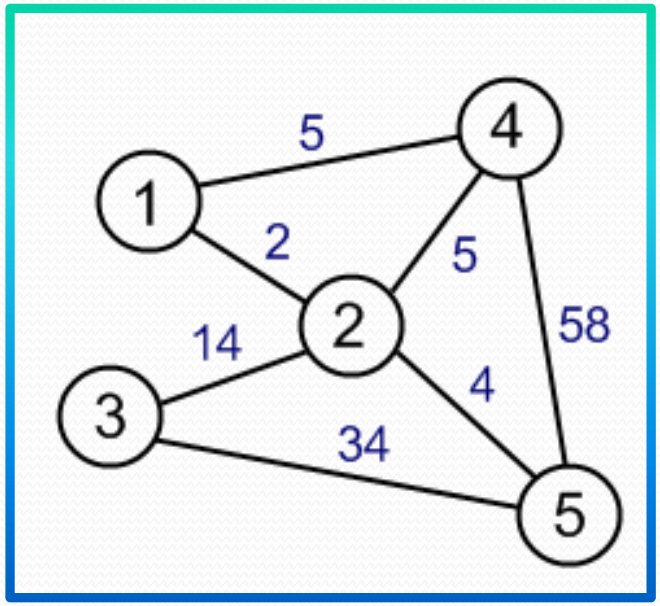
Fig:2

- **Directed Graphs:**



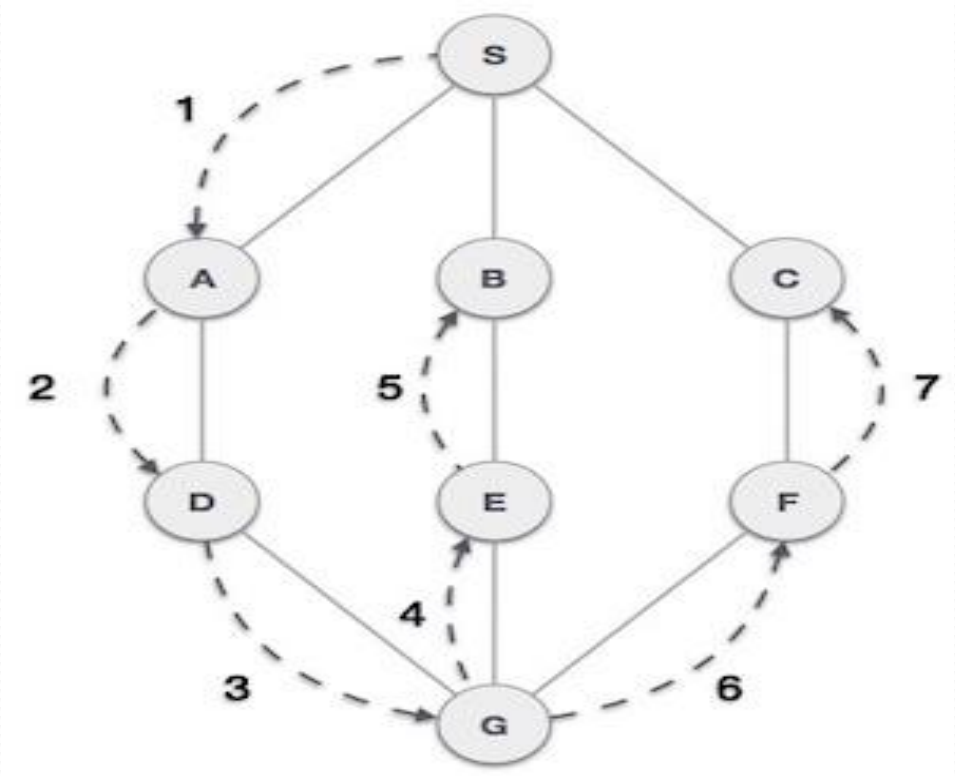
Types of Graphs:

- **Weighted Graph:**



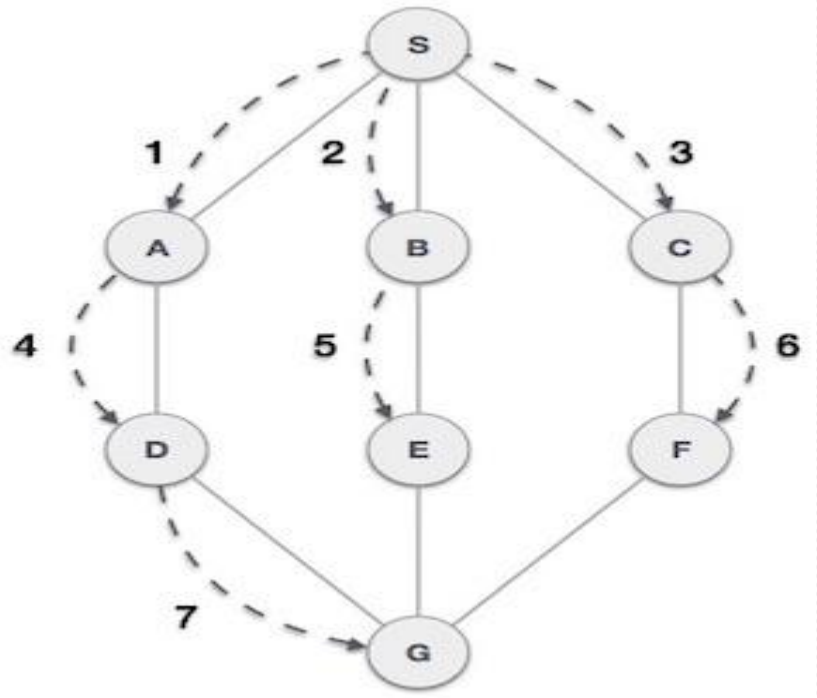
Depth First Traversal:

- Depth First Search (**DFS) algorithm** traverses a graph in a **depthward motion** and uses a **stack** to remember to get the next vertex to start a search, when a dead end occurs in any iteration.



Breadth First Traversal

- Breadth First Search (**BFS) algorithm** traverses a graph in a **breadthward motion** and uses a **queue to remember** to get the next vertex to start a search, when a dead end occurs in any iteration.

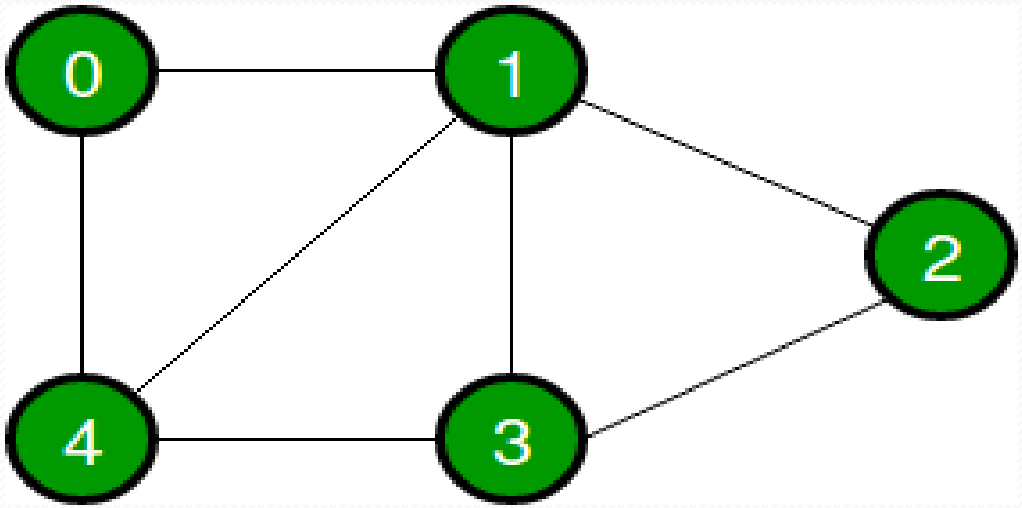


Graph and its representations:

- A graph is a data structure that consists of the following two components:
  1. A finite set of vertices also called as nodes.
  2. A finite set of ordered pair of the form (u, v) called as edge.
- Graphs are used to represent **many real-life applications**: Graphs are used to **represent networks**.
- The networks may include paths in a city or telephone network or circuit network. Graphs are also used in **social networks like linkedIn, Facebook.**
- For example, in Facebook, each person is represented with a vertex(or node).
- Each node is a structure and contains information like person id, name, gender, and locale.

Graph and its representations:

- Following is an example of an **undirected graph with 5 vertices.**



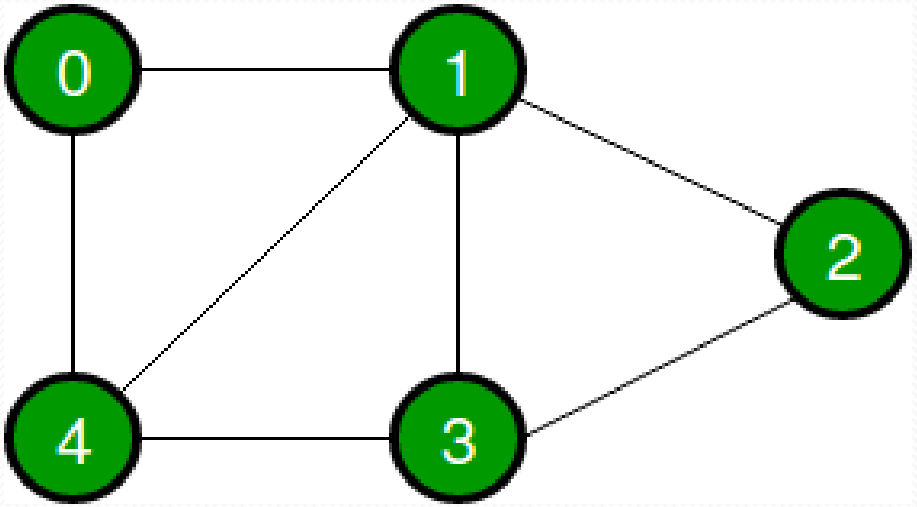
The following two are the most commonly used representations of a graph.

1. Adjacency Matrix

2. Adjacency List

Adjacency Matrix:

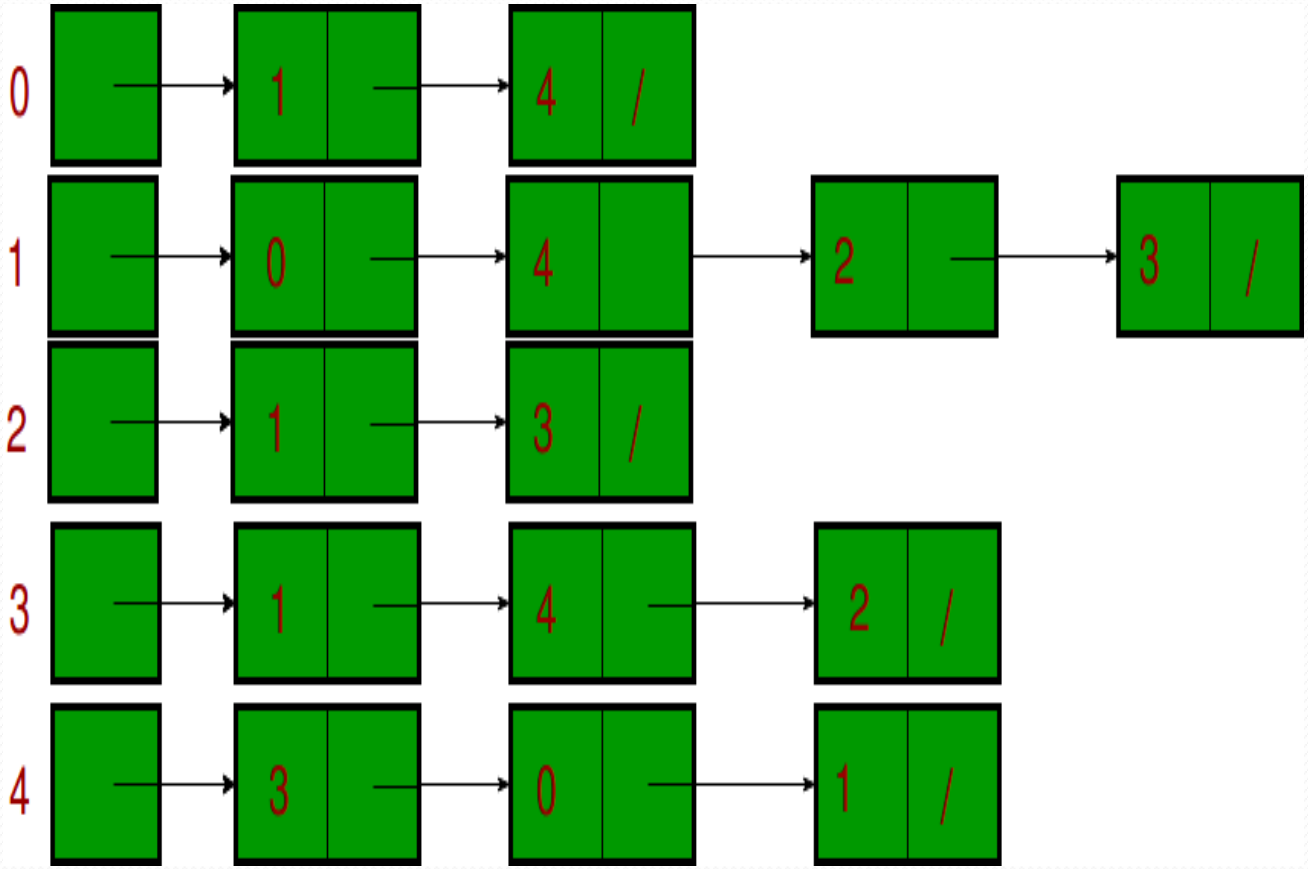
- Adjacency Matrix is a 2D array of size V x V where V is the number of vertices in a graph.
- The adjacency matrix for the above example graph is:

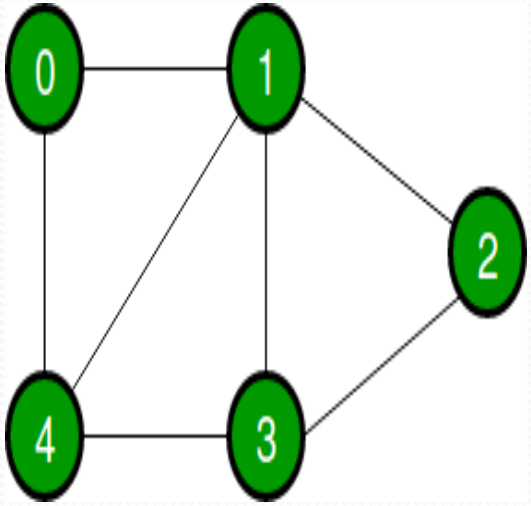


	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

Adjacency List:

- An **array** of lists is used. The size of the array is equal to the number of vertices. Let the array be an array[].





Graph Traversals

- BFS , Breadth first search
- DFS , Depth first search

Breadth First Search (BFS)

What is BFS?

Breadth First Search (BFS) is a graph traversal technique that explores nodes **level by level** (i.e., all neighbors first before going deeper).

It uses:

Queue (FIFO) data structure

Key Idea

Start from a node → visit it → visit all its neighbors → then move to the next level

Algorithm (Steps)

1. Start from a source vertex.
2. Mark it as **visited** and enqueue it.
3. While the queue is not empty:
   - Dequeue a node.
   - Visit all its unvisited adjacent nodes.
   - Mark them visited and enqueue them.

```
int adj[MAX][MAX]; // Adjacency Matrix
bool visited[MAX]; // Visited array
int n; // Number of vertices

int main() {
cout << "Enter number of vertices: ";
cin >> n;
cout << "Enter adjacency matrix:\n";
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
cin >> adj[i][j];
}
}

// Initialize visited array
for (int i = 0; i < n; i++) {
visited[i] = false;}

cout << "BFS Traversal: ";
bfs(0); // Start from vertex 0

return 0;}
```

```
// BFS function
    void bfs(int start)
 {
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty())
{
            int node = q.front();
            q.pop();
            cout << node << " “;
            for (int i = 1; i <= n; i++) {
                        if (adj[node][i] == 1 && !visited[i])
                        {
                        visited[i] = true;
                        q.push(i);
                        }
            }
    } // end while
    } // end bfs
```

Depth First Search (DFS)

What is DFS?

Depth First Search (DFS) is a graph traversal technique that explores a graph **as deep as possible along each branch before backtracking**.

It uses:

Stack (LIFO) concept

Key Idea

Start from a node → visit it → go to an unvisited neighbor → repeat → if no neighbor, **backtrack**

Algorithm (Steps)

1. Start from a source vertex.
2. Mark it as **visited**.
3. Visit all adjacent unvisited vertices recursively.
4. Backtrack when no more unvisited nodes are available.

```
#int adj[MAX][MAX];   // Adjacency Matrix
bool visited[MAX];   // Visited array
int n;               // Number of vertices
int main() {
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter adjacency matrix:\n";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
      {
            cin >> adj[i][j];
        }
    }
    // Initialize visited array
    for (int i = 0; i < n; i++) {
        visited[i] = false;     }
    cout << "DFS Traversal: ";
    dfs(0);  // Start DFS from vertex 0
    return 0;
}
```

```
// DFS function, Non-recursive
void dfs(int start) {
stack<int> st;
st.push(start);
while (!st.empty()) {
        int node = st.top();
        st.pop();
                if (!visited[node])
                {
                        cout << node << " ";
                        visited[node] = true;
                        // Push adjacent vertices (in reverse order)
                        for (int i = n-1 ; i > =0; i--) {
                                if (adj[node][i] == 1 && !visited[i]) {
                                st.push(i); }
                        }
                }
} // end while
} // end dfs
```